Clight2Rust: A Lightweight Framework for Translating a Subset of C to Safer Rust

Dongwen Yao, Jinhua Wu, and Yuting Wang

Shanghai Jiao Tong University

TASE 2026, July 4–6, 2026
Fairfield Hotel, Shanghai, China

Main Thesis

Contributions and Outcome

- We present Ptr<T>: a uniform abstraction that bridges C pointers to Rust.
- We confines unsafe operations to a small trusted runtime and FFI boundary.
- We implements a C $\rightarrow$ Rust translation pipeline on top of CompCert.
- We achieves a **100%** translation success rate on the CompCert benchmark suite, with an average overhead of **1.70x**.

A Counterintuitive Example

This C code is entirely valid

```c
int *p = malloc(2 * sizeof(int));
int *q = p;      // alias
q[0] = 1;
return p[0];     // expects 1
```

Attempt 1: Move Error

```rust
let p = vec![0; 2].into_boxed_slice();
let mut q = p; // Ownership moved!
q[0] = 1;
return p[0];   // Compile Error:
               // use of moved value `p`
```

Attempt 2: Logic Error

```rust
let p = vec![0; 2].into_boxed_slice();
let mut q = p.clone(); // Deep copy
q[0] = 1;
return p[0];   // Logic Error: returns 0,
               // p is not modified!
```

Why the Field Is Difficult

If the goal is only "make it run"

- One can translate into raw pointers.
- One can wrap the result in `unsafe`.
- Behavioral fidelity may be preserved, but most of Rust's safety benefit disappears.

If the goal is only "maximal static safety"

- One needs strong alias, region, and lifetime reasoning.
- Complex pointer arithmetic and runtime offsets quickly become difficult.
- Valid C programs may still be rejected.

Consequence

Such tools often produce "C written in Rust syntax".

Consequence

Safety improves, but automation and coverage often suffer.

Limitations of Existing Directions

Syntactic transpilation

Representative: C2Rust
Strength: broad coverage
Limitation: extensive `unsafe`

Purely static methods

Representative: Laertes, Split Tree-style work
Strength: closer to safe Rust
Limitation: hard cases remain difficult to prove statically

LLM-based approaches

Representative: *Gpt5, Gemini*
Strength: often more idiomatic Rust
Limitation: weak high-assurance guarantees

Position of This Work

This work starts from a verified compiler infrastructure and uses a small trusted runtime to obtain stronger automation and wider translatability.

How the Rest of the Talk Supports the Main Thesis

- First, I explain why `Ptr<T>` preserves **aliasing**, **pointer arithmetic**, and **lifetime extension**.
- Next, I show how the translation rules turn implicit C memory actions into explicit Rust operations.
- Finally, I use the experimental results to show that this bridge is practically viable.

Thus

The examples, the implementation, and the evaluation all answer the same question: is this semantic bridge actually workable?

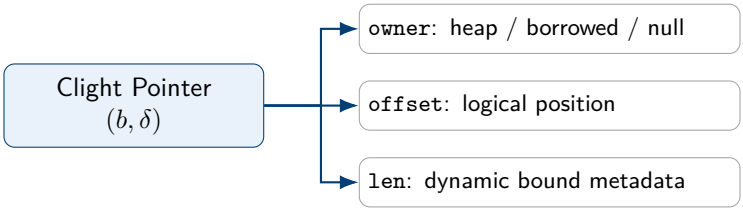
The Central Technical Problem

1. A C pointer acts both as an **ownership handle** and as a **movable cursor**.
2. Rust has no single safe primitive that directly captures both roles.
3. If we translate into native borrowing, many programs fail to compile.
4. If we fall back to raw pointers, the safety objective is lost.

Technical Answer

The key idea is to resolve C-to-Rust pointer conversion through a runtime-backed abstraction: `Ptr<T>` combines shared ownership, reference counting, and dynamic borrow checking, so that complex C pointer behavior can be translated into safe Rust operations without losing aliasing or pointer-arithmetic semantics.

Ptr<T> as a Semantic Wrapper for C Pointers



Intuition

- `owner`: backing storage
- `offset`: current element
- `len`: valid access range

Design payoff

- movement preserves ownership
- aliases share one storage object
- access goes through explicit checks

How Ptr<T> Is Implemented

```
enum Owner<T> {
    Heap(SharedBuffer<T>),
    Borrowed(BaseAddr, Size),
    Null,
}

struct Ptr<T> {
    owner: Owner<T>;
    offset: isize;
    len: usize;
}
```

Three fields

- `owner`: who owns the storage
- `offset`: where the pointer is
- `len`: how far access may go

Core insight

- aliases share one runtime owner
- pointer arithmetic updates only `offset`
- checks are centralized in the runtime

Core Operations of Ptr<T>

```
alloc(n):
    owner  <- shared(new array[n])
    offset <- 0
    len    <- n

clone(p):
    return Ptr(share(p.owner), p.offset, p.len)

offset(p, k):
    return Ptr(p.owner, p.offset + k, p.len)

load/store(p, i):
    assert p != null
    assert in_bounds(p.offset + i, p.len)
    access p.owner[p.offset + i]
```

Why it works

- sharing models C-style aliasing
- `offset` models pointer movement
- runtime checks replace undefined behavior

Translation reading

- `q = p` ⇒ `q = p.clone()`
- `p + i` ⇒ `p.offset(i)`
- `*p` ⇒ checked `load/store`

How C Pointer Operations Become Rust Operations

C construct	Meaning in C	Translation idea
`p = malloc(n)`	create heap object	`p = Ptr::alloc(n)`
`q = p`	create alias	`q = p.clone()`
`p + i`	move pointer	`p.offset(i)`
`*p`	dereference for read	`p.load(0)`
`*p = v`	dereference for write	`p.store(0, v)`
`p == NULL`	null test	`p.is_null()`

Main message

implicit pointer behavior in C is turned into explicit method calls in Rust.

Why This Abstraction Matters

It preserves three essential C behaviors

- **Aliasing**: multiple pointers may observe one mutable object
- **Pointer arithmetic**: movement does not lose the original allocation
- **Lifetime extension**: heap data may outlive inner scopes

It also satisfies Rust-side requirements

- generated logic remains safe Rust
- dangerous behavior is isolated in the runtime
- invalid access becomes an explicit panic

Key Point

The real accomplishment of the paper is not the invention of yet another pointer type, but the identification of a small, safer, and practical semantic bridge.

Example 1: Mutable Aliasing

```
int *p = malloc(2 * sizeof(int));
int *q = p; // q aliases p
q[0] = 1;
return p[0];
```

Why this is difficult

- C permits multiple mutable views of the same memory.
- Rust's `&mut` model does not permit that directly.
- A naive use of `Box` or ordinary cloning destroys the sharing semantics.

Translation result

```
let mut p: Ptr<i32> = Ptr::alloc(2);
let mut q = p.clone();
q.store(0, 1);
return p.load(0);
```

Example 2: Pointer Arithmetic Must Be Separated from Ownership

```
int *p = malloc(2 * sizeof(int));
p = p + 1;
*p = 42;
```

Difficulty

- In C, the pointer itself may move.
- However, the ownership information for the original allocation must not be lost.
- A slice- or borrow-based encoding may preserve only the shifted view and lose the original allocation identity.

Translation result

```
let mut p: Ptr<i32> = Ptr::alloc(2);
p = p.offset(1);
p.store(0, 42);
```

Why Purely Static Reasoning Fails on Dynamic Offsets

Original C Code:

```c
void demo(int *arr, int k) {
    int *p1 = arr + 0;
    int *p2 = arr + 5;
    *p1 = 10;
    *p2 = 20;
    int *p3 = p1 + k;
    *p3 = *p1 + *p2;
}
```

The Fundamental Difficulty

- **Static:** p1/p2 locations are compile-time known.
- **Dynamic:** p3 destination depends on runtime k.
- **Overlap:** Compiler cannot statically check aliasing.

Implication

Static splitting alone is insufficient. Runtime safety is necessary.

Translation Result (Rust)

```rust
let mut p1: Ptr<i32> = arr.clone().offset(0);
let mut p2: Ptr<i32> = arr.clone().offset(5);
p1.store(0, 10);
p2.store(0, 20);
let mut p3: Ptr<i32> = p1.clone().offset(k as isize);
p3.store(0, p1.load(0) + p2.load(0));
```

Pipeline Architecture

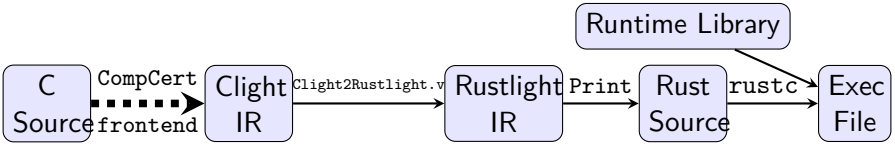


Figure: End-to-end translation pipeline of Clight2Rust

Two Key Files in the Repository

`clight2rustlight/Clight2Rustlight.v` defines the translation, and `rustfrontend/PrintRustlight.ml` prints Rustlight as compilable Rust code.

What Do the Experimental Results Show?

Setup

- Benchmark suite: CompCert `test/c`
- Reference baseline: C programs compiled by Clang/GCC
- The subset of C supports: supports all of the MISRA-C 2004 subset of C

Headline result

- 24/24 benchmarks are translated and executed successfully
- Output matches the reference C implementations
- Average runtime overhead is approximately 1.70x

Metric	Value	Interpretation
Translation success rate	100%	all benchmark programs compile and run successfully
Average overhead	1.70x	memory safety is achieved with a measurable runtime cost
Fastest case	0.67x	LLVM optimization may outperform the reference C build

How Should We Interpret the 1.70x Overhead?

Main sources of overhead

- the uniform `load/store` interface
- dynamic bounds checking
- dynamic borrow tracking in `Rc<RefCell<...>>`

Why the result still matters

- It demonstrates that the lightweight runtime-backed route is viable.
- The cost is noticeable, but not prohibitive.
- More importantly, the work achieves a meaningful balance among safety, automation, and semantic credibility.

Conclusion

Takeaway

The contribution of this paper is not simply the construction of another C-to-Rust translator. Its central achievement is to identify and address the most difficult problem: how to migrate C pointer semantics into a form that safe Rust can meaningfully represent.

- **Contribution:** it reframes the task as memory-semantic migration.
- **Innovation:** it proposes a practical answer based on `Ptr<T>` and a small trusted runtime.
- **Significance:** it connects verified compiler infrastructure with Rust's safety discipline.
- **Future work:** it opens the way to stronger static reasoning, including Split Tree-style ideas.

Thank you.